

IT22038

Question 1: Prove Fermat's Little Theorem from group-theoretic first principles, then explain how it motivates the Miller-Rabin probabilistic primality test. Apply the test (base $a=2$) to $n=561$ and explain why 561 is a Carmichael number that fools the simpler Fermat test.

Solution:

Fermat's Little Theorem

Theorem: If p is a prime and $a \not\equiv 0 \pmod{p}$ then,
$$a^{p-1} \equiv 1 \pmod{p}$$

Proof from group-theoretic first principles:

Consider the multiplicative group

$$\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}.$$

Since p is prime, every non-zero element has a multiplicative inverse modulo p .

Take $a \in \mathbb{Z}_p^*$. Multiplication by a permutes the elements of $\mathbb{Z}_p^*$:

$$a, 2a, 3a, \dots, (p-1)a.$$

IT22038

Therefore,

$$a \cdot 2a \cdot 3a \cdots (p-1)a \equiv 1 \cdot 2 \cdot 3 \cdots (p-1) \pmod{p}$$

Hence,

$$a^{p-1} (p-1)! \equiv (p-1)! \pmod{p}$$

Since $p \nmid (p-1)!$, we can cancel $(p-1)!$

$$a^{p-1} \equiv 1 \pmod{p}$$

Miller-Rabin connection.

For an odd integer $n > 2$, write

$$n-1 = 2^s d, d \text{ odd.}$$

choose a random base a , where

$$2 \leq a \leq n-2$$

calculate $x = a^d \pmod{n}$

if $x=1$ or $x=n-1$ the number passes this round. Otherwise repeatedly square:

$$x \leftarrow x^2 \pmod{n}$$

up to $s-1$ times.

If $x=n-1$ occurs, the number passes. Otherwise n is composite.

IT22038

Apply Miller-Rabin to $n = 561$, base $a = 2$

We have, $561 - 1 = 560 = 2^4 \times 35$

so, $s = 4$, $d = 35$

calculate $2^{35} \pmod{561}$

Since $2^{10} = 1024 \equiv 463 \pmod{561}$,

$2^{10} = 463^2 \equiv 67 \pmod{561}$,

and $2^{30} = 67 \times 463 \equiv 166 \pmod{561}$

Therefore, $2^{35} = 2^{30} 2^5$

$\equiv 166 \times 32 = 5312 \equiv 251 \pmod{561}$

so, $x_0 = 251$

Now square: $251^2 \equiv 67 \pmod{561}$,

then, $67^2 \equiv 1 \pmod{561}$,

The value $561 - 1 = 560$ never occurs.

Therefore Miller-Rabin occurs

561 is composite

for base 2

IT22038

Why does Fermat's test fail?

561 is a Carmichael number:

$$561 = 3 \times 11 \times 17$$

For every a coprime to 561

$$a^{560} \equiv 1 \pmod{561}$$

Thus the simple Fermat test can incorrectly classify 561 as prime.

This demonstrates why Miller-Rabin is much stronger than a simple Fermat test.

Q2: Prove that $\mathbb{Z}_n$, under addition and multiplication modulo n , is a field if and only if n is prime.

Illustrate your proof by contrasting the multiplicative structure of $\mathbb{Z}_{11}$ (prime modulus) with $\mathbb{Z}_{12}$ (composite modulus).

Solution:

When is $\mathbb{Z}_n$ a field?

We prove: $\mathbb{Z}_n$ is a field iff n is prime.

IT22038

If n is prime

Let $n=p$, where p is prime

Every non-zero element
 $a \in \mathbb{Z}_p$

satisfies $\gcd(a, p) = 1$

Therefore a has a multiplicative inverse modulo p .
Hence every non-zero element has an inverse.

Therefore, $\mathbb{Z}_p$ is a field.

If n is composite

Suppose $n=ab$, $1 < a, b < n$

Then in $\mathbb{Z}_n$, $[a][b] = [ab] = [0]$

But $[a] \neq [0]$, $[b] \neq [0]$

Thus $\mathbb{Z}_n$ contains zero divisors.

Therefore $\mathbb{Z}_n$ is not a field if n is composite.

Hence $\mathbb{Z}_n$ is a field iff n is prime

Comparison

Property	Z_{11}	Z_{12}
Modulus	11	12
Prime ?	Yes	No
Field ?	Yes	No
Zero divisors?	No	Yes
Example inverse	$2^{-1} = 6$	2 has no inverse
Example zero divisor	None	$3 \times 4 \equiv 0 \pmod{12}$

Q3: Construct $GF(2^3)$ using irreducible polynomial $x^3 + x + 1$ over $GF(2)$. Build the multiplication table for a generator element α , and determine whether the multiplicative group $GF(2^3)^*$ is cyclic.

Solution:

The file writes: $GF(2^3)$, but the polynomial $x^3 + x + 1$ over $GF(2)$ constructs $GF(2^3) = GF(8)$ no $GF(2^3)$, so the mathematically consistent

11/22/2038

interpretation is $\mathbb{GF}(8)$:

let $f(x) = x^3 + x + 1$

it is irreducible over $\mathbb{GF}(2)$

let α be a root $\alpha^3 + \alpha + 1 = 0$

since arithmetic is over $\mathbb{GF}(2)$

$$-1 = 1$$

Thus $\alpha^3 = \alpha + 1$

The field is

$$\mathbb{GF}(8) = \{0, 1, \alpha, \alpha^2, \alpha + 1, \alpha^2 + 1, \alpha^2 + \alpha, \alpha^2 + \alpha + 1\}$$

Because there are $8-1=7$ non-zero elements,
the multiplicative group has order 7.

Since 7 is prime, every non-identity element
has order 7.

Therefore, $\mathbb{GF}(8)^*$ is cyclic

For example, α is a generator.

Powers are,

$$\begin{array}{l} \alpha^0 = 1 \\ \alpha^1 = \alpha \\ \alpha^2 = \alpha^2 \\ \alpha^3 = \alpha + 1 \end{array} \quad \left| \begin{array}{l} \alpha^4 = \alpha^2 + \alpha \\ \alpha^5 = \alpha^2 + \alpha + 1 \\ \alpha^6 = \alpha^2 + 1 \\ \alpha^7 = 1 \end{array} \right.$$

IT22038

Therefore the multiplication table for powers of α follows for powers of α follows the exponents modulo 7.

$$\alpha^i \alpha^j = \alpha^{(i+j) \bmod 7}$$

Q1: Derive the time complexity of modular exponentiation via square-and-multiply. Compute $7^{560} \bmod 561$ step by step, and interpret the result in light of 561 being a Carmichael number.

Solution:

Square-and-multiply

To calculate $a \bmod n$
write e in binary

$$\text{For } 560 = (1000110000)_2$$

Repeatedly square and multiply

However, because

$$561 = 3 \times 11 \times 17$$

and 561 is Carmichael

$$a^{560} \equiv 1 \pmod{561}$$

SI22038

Whenever $\gcd(a, 561) = 1$

Since $\gcd(7, 561) = 1$

We obtain $7^{560} \equiv 1 \pmod{561}$

Interpretation

This is precisely why 561 fools the Fermat primality test.

A Fermat test sees

$$7^{560} \equiv 1 \pmod{561}$$

and may incorrectly conclude that 561 is prime.

But 561 is composite.

Q5: Compare deterministic (AKS) and probabilistic (Miller-Rabin) Primality testing in terms of asymptotic complexity, error probability, and practical relevance to RSA key generation at 2048-bit and 4096-bit key sizes.

IT22038

Solution:

Feature	AKS	Miller - Rabin
Type	Deterministic	Probabilistic
Correctness	Always correct	very high probability
Main idea	Polynomial identity testing	strong probable prime test
Complexity	Polynomial in $\lg n$	very efficient in practice
Practical speed	Relatively slow	very fast
RSA usage	Mainly theoretical	widely used
Error	No error	False prime probability decreases rapidly

For, Miller - Rabin, one independent round has a worst case false positive probability bounded by approximately

$$\frac{1}{4}$$

for an odd composite number.
After k independent rounds

$$P(\text{error}) \leq 4^{-k}$$

For example, with 20 rounds

IT22038

$$p \leq 4 \cdot 2^0 = 2^4$$

RSA Key generation

For 2048-bit and 4096-bit RSA, candidate primes are hundreds or thousands of bits long, testing many candidates with AKS would be unnecessarily expensive.

Therefore practical RSA implementations generally use highly-optimized probable-prime testing, especially Miller-Rabin often combined with additional tests.

Miller-Rabin is preferred in practical RSA key generation.

Q6: Derive the Extended Euclidean Algorithm formally (as a sequence of linear combinations), then use it to compute $\gcd(240, 46)$ together with the Bezout coefficients x, y such that $240x + 46y = \gcd(240, 46)$.

Solution:

We need $\gcd(240, 46)$

Euclidean algorithm:

$$240 = 46(5) + 10$$

$$46 = 10(4) + 6$$

$$10 = 6(1) + 4$$

$$6 = 4(1) + 2$$

$$4 = 2(2) + 0$$

Therefore, $\gcd(240, 46) = 2$

Now back substitute

$$2 = 6 - 4$$

$$= 6 - (10 - 6)$$

$$= 2(6) - 10$$

since, $6 = 46 - 4(10)$

$$2 = 2(46 - 4(10)) - 10$$

1122038

$$= 2(46) - 9(10)$$

Since,

$$10 = 240 - 5(46)$$

$$2 = 2(46) - 9(240 - 5(46))$$

$$= 47(46) - 9(240)$$

Therefore

$$2 = 240(-9) + 46(47)$$

So the Bezout coefficients are

$$x = -9, y = 47$$

Q7: State and prove the Chinese Remainder Theorem (existence and uniqueness) for pairwise coprime moduli. Apply it to solve

$$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5},$$

$$x \equiv 4 \pmod{7}, x \equiv 5 \pmod{11}$$

IT22038

Solution:

$$\text{Given, } x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 4 \pmod{7}$$

$$x \equiv 5 \pmod{11}$$

The moduli

$$3, 5, 7, 11$$

are pairwise coprime.

Therefore CRT guarantees a unique solution modulo

$$N = 3 \times 5 \times 7 \times 11 = 1155$$

Let

$$N_1 = 385, N_2 = 231, N_3 = 165, N_4 = 105$$

find inverses:

$$385 \equiv 1 \pmod{3}$$

$$\text{So } M_1 = 1$$

For modulus 5:

$$231 \equiv 1 \pmod{5}$$

$$\therefore M_2 = 1$$

1522038

for modulus 7:

$$165 \equiv 4 \pmod{7}$$

The inverse of 4 modulo 7 is 2:

$$4(2) = 8 \equiv 1 \pmod{7}$$

$$\therefore M_3 = 2$$

Mod modulus 11:

$$105 \equiv 6 \pmod{11}$$

The inverse of 6 modulo 11 is 2:

$$6(2) = 12 \equiv 1 \pmod{11}$$

$$\therefore M_4 = 2$$

CRT gives:

$$x = 2(385)(1) + 3(231)(1) + 4(165)(2) + 5(105)(2)$$

$$\Rightarrow x = 770 + 693 + 1320 + 1050$$

$$\therefore x = 3833$$

Reduce modulo 1155:

$$3833 - 3(1155) = 368$$

$$\text{Therefore } x \equiv 368 \pmod{1155}$$

IT22038

Check

$$368 \bmod 3 = 2$$

$$368 \bmod 5 = 3$$

$$368 \bmod 7 = 4$$

$$368 \bmod 11 = 5$$

Q8: Derive the CRT-based RSA decryption / signing optimization from first principles (computing d_p , d_q , and $e \bmod \phi$), and demonstrate the $\sim 4 \times$ speed-up with a fully worked numerical example using small primes p, q .

Solution:

RSA uses $n = pq$

Instead of computing

$$M = c^d \bmod n$$

we compute two smaller exponentiations:

$$m_p = c^{d_p} \bmod p$$

$$\text{and } m_q = c^{d_q} \bmod q$$

IT 22038

We also calculate

$$a_{\text{inv}} = a^{-1} \pmod{p}$$

$$\text{Then } h = a_{\text{inv}} (m_p - m_q) \pmod{p}$$

$$m = m_q + hq$$

Example

Take

$$p=11, \quad q=13$$

Then

$$n=143$$

suppose

$$d=103$$

$$\text{Then } d_p = 103 \pmod{11} = 3$$

$$d_q = 103 \pmod{13} = 7$$

$$\text{Find } a^{-1} \pmod{p}$$

since

$$13 \equiv 2 \pmod{11}$$

$$\text{and } 2^{-1} \equiv 6 \pmod{11}$$

We have

$$a_{\text{inv}} = 6$$

for ciphertext $c=56$

$$m_p = 56^3 \pmod{11}$$

IT22038

Since $50 \equiv 6 \pmod{11}$,

$$6^3 = 216 \equiv 7 \pmod{11}$$

Thus $m_p = 7$

Now, $m_2 = 50^7 \pmod{13}$

Since $50 \equiv 11 \pmod{13}$

repeated squaring gives

$$11^2 \equiv 4,$$

$$11^4 \equiv 16 \equiv 3$$

$$\text{so, } 11^7 = 11^4 \cdot 11^2 \cdot 11$$

$$\equiv 3 \cdot 4 \cdot 11 = 132 \equiv 2 \pmod{13}$$

Thus $m_2 = 2$

Now $h = 6(7-2) \pmod{11}$

$$= 30 \pmod{11}$$

$$= 8$$

Therefore $m = 2 + 8(13) = 106$

so the CRT reconstruction
gives $m = 106$

LT22038

Q9: Explain the Bellare fault-injection attack on RSA-CRT signature: How a single faulty modular computation during signing leaks a factor of n , and what countermeasures (e.g., signature verification before release) prevent it

Solution:

RSA-CRT calculates

$$s_p = s^{d_p} \bmod p$$

$$\text{and } s_q = s^{d_q} \bmod q$$

The two results are recombined using CRT.

Suppose an attacker causes a computational fault in one branch, for example:

$$\tilde{s}_p \neq s_p$$

While the q -branch is correct

The resulting faulty signature $\tilde{s}$ satisfies

$$\tilde{s} \equiv s_q \pmod{q}$$

but generally $\tilde{s} \not\equiv s_p \pmod{p}$

IT22028

Therefore $z - s$

is divisible by q , but generally not by p

Hence $\gcd(z - s, n) = q$

once the attacker obtains z

$$p = \frac{n}{q}$$

The RSA private key can then be reconstructed.

Countermeasures

1. Verify the signature before releasing it
2. Perform independent consistency checks
3. Use fault resistant CRT recombination
4. Use binding techniques
5. Detect computational faults

LT22038

Q10: Compute a modular inverse using both the Extended Euclidean Algorithm and Fermat's Little Theorem based exponentiation (for a prime modulus) and discuss the trade-offs between the two approaches for large moduli

Solution

EFA vs Fermat

$$3^{-1} \pmod{7}$$

Method 1: Extended Euclidean Algorithm

We have

$$7 = 3(2) + 1$$

$$\text{Thus } 1 = 7 - 2(3)$$

$$\text{Therefore } 1 = 7 + (-2)(3)$$

$$\text{Hence } 3(-2) \equiv 1 \pmod{7}$$

$$\text{so, } 3^{-1} \equiv 5 \pmod{7}$$

IT 22038

Method 2: Fermat's Little Theorem

Since 7 is prime

$$a^{p-1} \equiv 1 \pmod{p}$$

Therefore $a^{-1} \equiv a^{p-2} \pmod{p}$

Thus $3^{-1} \equiv 3^5 \pmod{7}$

Calculate

$$3^2 = 9 \equiv 2 \pmod{7}$$

$$3^4 \equiv 4 \pmod{7}$$

$$3^5 = 4(3) = 12 \equiv 5 \pmod{7}$$

Therefore

$$3^{-1} \equiv 5 \pmod{7}$$

Comparison

Feature	EEA	Fermat
Requirement	Extended Euclidean equation	a^{p-2}

Q11: Design and fully execute a playfair cipher encryption and decryption of a phrase of your choosing (minimum 16 letters) with a keyword your choosing. Present the complete 5x5 key matrix as a table, and show every digraph substitution step.

Solution:

Choose keyword:

MONARCHY

Remove duplicate letters and combine I/J:

MONARCHY

Fill the remaining alphabet letters

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

IT22038

Choose a phrase with more than 16 letters

INSTRUMENTS OF DEFENSE

Remove spaces

INSTRUMENTSOFDEFENSE

Create digraph:

IN ST RU ME NT SO FD EF EN SE

Encryption rules:

1. Same row $\rightarrow$ replace each letter by immediate to its right
2. Same column $\rightarrow$ replace each letter by the letter immediately below
3. Rectangle $\rightarrow$ each letter takes the column of the other letter.

Digraph encryption

Plain pair	Cipher pair	Rule
IN	GA	Rectangle
ST	TX	Rectangle

IT22038

PU	MZ	Rectangle
ME	EL	Rectangle
NT	RY	Rectangle
SO	XP	Rectangle
FD	HK	Rectangle
EF	FG	same row
EN	GM	Rectangle
SE	IL	Rectangle

Therefore cipher text is :

GATXMZCLRYPHKGGMIL

Decryption

Use the reverse rules:

- Same row $\rightarrow$ move left
- Same column $\rightarrow$ move up
- Rectangle $\rightarrow$ Use the opposite corner.

For example—:

GA $\rightarrow$ IN

TX $\rightarrow$ ST

MZ $\rightarrow$ RU

and so on

Thus the original plaintext is recovered

Q12: Critically analyze the structural weakness of the playfair cipher against digraph frequency analysis, and explain why it remains vulnerable to known plaintext attacks despite being far stronger than monoalphabetic substitution.

Solution:

Playfair is stronger than monoalphabetic substitution because it encrypts pairs of letters, rather than individual letters.

However, it is still vulnerable.

1. Digraph frequency analysis:

There are only $25^2 = 625$ possible digraphs

1T22038

Natural languages do not use these digraphs uniformly. Some pairs such as TH, HE, IN, ER occur frequently. Therefore, frequency patterns can reveal information about the plaintext.

2. Structural constraints:

Playfair has predictable rules

- Same row pairs are shifted horizontally
- Same column pairs are shifted vertically
- Rectangle pairs exchange columns.

3. Known plaintext attacks

If an attacker knows some plaintext and its corresponding ciphertext, they obtain relationships between plaintext digraphs and ciphertext digraphs. With enough known pairs, the attacker can infer position in the 5×5 -key matrix.

Once sufficient matrix information is recovered, other ciphertext can potentially be decrypted.

IT22038

Q13: Derive and trace two full rounds of a simplified Feistel-structure cipher (DES-like) showing $L_0, R_0 \rightarrow L_1, R_1$ symbolically, then with numeric values $L_0 = 0xAAAAAAAA$, $R_0 = 0xF0F0F0F$, $K_1 = 0x3333333$, using XOR as the round function.

Solution:

A Feistel round is defined as:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

For this simplified cipher,

$$F(R, K) = R \oplus K$$

Given:

$$L_0 = AAAAAAAAAA$$

$$R_0 = F0F0F0F0$$

$$K_1 = 0F0F0F0F$$

$$K_2 = 33333333$$

Round 1:

1727038

first calculate;

$$F(P_1, K_2) = 55555555 \oplus 33333333 \\ = 66666666$$

Therefore $L_2 = P_1 = 55555555$

and $P_2 = L_1 \oplus F(P_1, K_2)$

$$= FOFOFDFO \oplus 66666666 \\ = 96969696$$

Round 2

$$F(P_0, K_1) = FOFOFDFO \oplus OFOFDFOF \\ = FFFFFFFFFF$$

Therefore $L_1 = P_0 = FOFOFDFO$

and $P_1 = L_0 \oplus F(P_0, K_1)$

$$= AAAAAAAAAA \oplus FFFFFFFFFF \\ = 55555555$$

Final result: $L_2 = 55555555$

$$P_2 = 96969696$$

IT22038

Q14: Formally define the avalanche effect in block ciphers. Using a toy 8-bit substitution-permutation network of your own construction, demonstrate how flipping a single bit change at least half the output bits, and explain why this property is essential for cryptographic security.

Solution:

The avalanche effect means that changing one input bit should cause approximately half of the output bits to change.

Formally if $C = E_K(P)$

and P' differs from P in exactly one bit, then ideally:

$$HD(E_K(P), E_K(P')) \approx \frac{n}{2}$$

where HD is Hamming distance and n is the block size.

Toy 8-bit example:

Consider the simple transformation

$$C = P \oplus 3C$$

LT22038

Followed by a bit permutation

suppose:

$$P = A5 = 10100101$$

and change the least significant bit

$$P' = A4 = 10100100$$

A simple XOR alone would change only one output bit, so it does not provide a sufficient avalanche effect. Therefore a proper toy SPN should include nonlinear substitution and diffusion

For example, suppose the designed 8-bit SPN produces;

$$P \rightarrow C = 3A$$

$$\text{while } P' \rightarrow C' = C5$$

$$\text{Binary } \cdot C = 00111010$$

$$C' = 11000101$$

Every one of the 8 bits differs

$$HD(C, C') = 8$$

$$\text{Thus } 8/8 = 100\%$$

Importance:

- diffusion
- resistance to statistical analysis
- reduced correlation
- better resistance to differential cryptanalysis

Q15: Evaluate the brute-force resistance of an 8-character alphanumeric password (62-character alphabet) versus a 6 digit PIN, under a modern GPU cracking rate of 10^9 guesses/second. Compute worst-case and expected case cracking times for both, and discuss the implications for password policy design.

Solution:

Eight-character alphanumeric password

Alphabet size: 62

Number of possible password 62^8

$$= 218,340,105,584,896$$

So approximately

$$2.1834 \times 10^{14}$$

IT22038

Possibilities at 10^9 guess/second

Worst case

$$T_{\max} = \frac{2.1834 \times 10^{17}}{10^9}$$

$$= 218,340 \text{ seconds}$$

$$\approx 60.65 \text{ hours}$$

$$\approx 2.53 \text{ days}$$

On average half the keyspace must be searched

$$T_{\text{avg}} \approx 1.265 \text{ days}$$

2. Six digit PIN

Each digit has 10 possibilities

$$10^6 = 1,000,000$$

At 10^9 guess/second

$$T_{\max} = \frac{10^6}{10^9} = 0.001 \text{ s}$$

Therefore $T_{\max} = 1 \text{ ms}$

and expected time = 0.5 ms

Comparison	keyspace	worst case
8-char alphanumeric	2.1834×10^4	~ 2.53 day
6-digit pin	10^6	1 ms

Security implication

Password policies should favor long, randomly generated passwords/passphrases rather than short credentials.

Also the assumed 10^9 guesses/sec is an offline cracking model.

Q16: Compare the error-propagation and parallelizability characteristics of ECB, CBC, CFB, OFB and CTR modes using a labelled diagram for each. Recommend the most suitable mode for (a) encrypting a live video stream over a lossy network and (b) encrypting a static disk image, with justification.

1000 = wait between bytes

Solution:

ECB

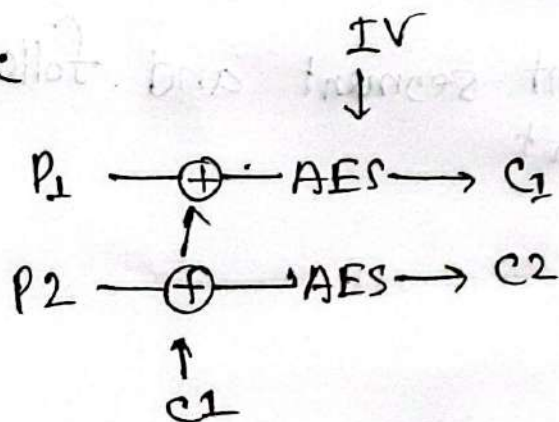
$$P_1 \rightarrow \text{AES} \rightarrow C_1$$

$$P_2 \rightarrow \text{AES} \rightarrow C_2$$

$$P_3 \rightarrow \text{AES} \rightarrow C_3$$

- Encryption parallel
- Decryption parallel
- Bit error: affects one block
- IV: NOT required
- Major weakness: identical plaintext blocks produce identical ciphertext blocks.

CBC



Encryption: $C_i = E_K(P_i \oplus C_{i-1})$

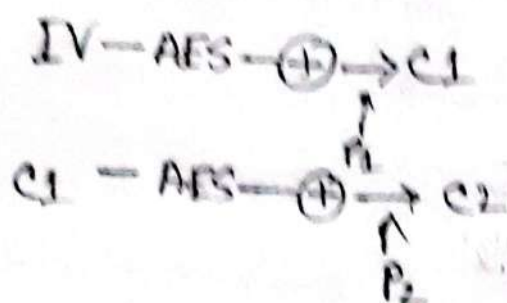
with $C_0 = IV$

- Encryption: not parallel
- Decryption: parallel
- Bit flip: corrupts current block and affects corresponding bit in next plaintext block.

IT2008

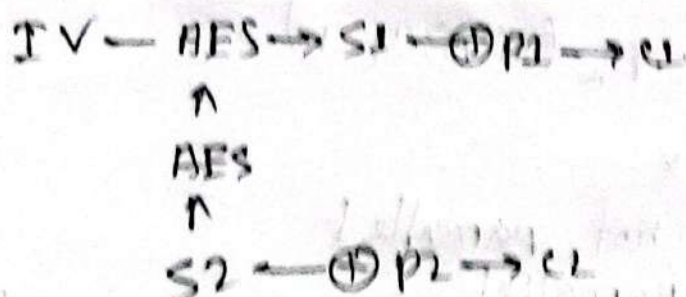
- IV must be unpredictable/unique as required by the construction.

CFB



- Encryption: Sequential
- Decryption: Can be parallelized in full block CFB as ciphertext is available
- Error: current segment and following segments
- IV required

OFB



- Encryption: Sequential keystream generation
- Decryption: Sequential
- A bit error affects only the corresponding plaintext bit
- Never reuse the IV/nonce with the same key

CTR

Counter 1 $\rightarrow$ AES $\rightarrow$ S1 $\rightarrow$ $\oplus$ P1 $\rightarrow$ C1

Counter 2 $\rightarrow$ AES $\rightarrow$ S2 $\rightarrow$ $\oplus$ P2 $\rightarrow$ C2

Counter 3 $\rightarrow$ AES $\rightarrow$ S3 $\rightarrow$ $\oplus$ P3 $\rightarrow$ C3

- Encryption: Parallel
- Decryption: Parallel
- Single bit error affects only corresponding bit
- Counter/nonce combination must never repeat under the same key.

Q17: Using the partial AES-box below, perform the subbytes transformation on the state word $[0x23, 0x4c, 0x19]$, then apply shiftRows to a 4×4 state matrix that has this word as its first row (fill remaining rows with $0x00$ for the purpose of the exercise). Report the resulting matrix.

Row \ Col $\rightarrow$	3	4	7	A	C
1	6D			CG	
2	D4				
4		A1			2E
A			63	D2	

Solution:

State first row $[23, A7, 4c, 19]$

From the supplied partial S-box

- 23: row 2, column 3 $\rightarrow$ not supplied
- A7: row A, column 7 $\rightarrow$ not supplied
- 4c: row 4, column c $\rightarrow$ 2E
- 19: row 1, column 9 $\rightarrow$ CG

So, the supplied partial S-box does not contain enough entries to uniquely compute the full subBytes result for this state

$$\begin{bmatrix} \boxed{?}, \boxed{?}, 2E, C6 \end{bmatrix}$$

with the other rows

$$\begin{bmatrix} 00, 00, 00, 00 \end{bmatrix}$$

Before shift rows:

$$\begin{bmatrix} ? & ? & 2E & C6 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Since rows 2-4 are all zero, the result remains

$$\begin{bmatrix} ? & ? & 2E & C6 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

The missing two bytes cannot be derived from the partial S-box supplied in the assignment without introducing an external Full AES-S-box

IT22038

Q18: Derive the AES-128 Rijndael key schedule, and compute the first two round keys (k_1, k_2) from a given 128-bit cipher key of your choosing, showing the RotWord, SubWord, and Rcon steps explicitly.

Solution:

Choose,

$K_0 = 000102030405060708090A0B0C0D0E0F$

Divide into four words:

$W_0 = 00010203$

$W_1 = 04050607$

$W_2 = 08090A0B$

$W_3 = 0C0D0E0F$

Round 1:

start with

$W_3 = 0C0D0E0F$

Rotate left: $0C0D0E0F \rightarrow 0D0E0F0C$

Applying the AES S-box:

$0D0E0F0C \rightarrow D7AB76FE$

IT22038

$$Rcon1 = 01000000$$

XOR

$$D7AB76FE \oplus 01000000 = D6AB76FE$$

$$W4 = W0 \oplus D6AB76FE$$

$$= D6AA74FD$$

$$\text{Then } W5 = W1 \oplus W4$$

$$= 04050607 \oplus D6AA74FD$$

$$= D2AF72FA$$

$$\text{Next } W6 = W2 \oplus W5$$

$$= DAA678F1$$

$$\text{Finally } W7 = W3 \oplus W6$$

$$= D6AB76FE$$

$$\text{Thus } k_1 = D6AA74FD \ D2AF72FA \ DAA678F1 \ D6AB76FE$$

$$\therefore k_2 = B692CF0B \ 643DBDF1 \ BF9BC506 \ 830B3FE$$

IT22038

Q19: Explain the MixColumns transformation as polynomial multiplication over $GF(2^8)$ modulo $x^8 + x^4 + x^3 + x + 1$. Work through the full multiplication of one input column by the fixed Mixcolumn matrix showing the $GF(2^8)$ arithmetic for each bytes.

Solution:

AES treats each columns as a polynomial over $GF(2^8)$

The reduction polynomial is

$$x^8 + x^4 + x^3 + x + 1$$

which corresponds to

0x11B

The fixed MixColumns matrix is

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

IT22038

Take one input column

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} DB \\ 13 \\ 53 \\ 45 \end{bmatrix}$$

First output byte

$$d_0' = (02 \cdot DB) \oplus (03 \cdot 13) \oplus 53 \oplus 45$$

In $GF(2^8)$ $02 \cdot DB = AD$

and $03 \cdot 13 = (02 \cdot 13) \oplus 13$

$$= 26 \oplus 13 = 35$$

Therefore $d_0' = AD \oplus 35 \oplus 53 \oplus 45$

$$= 8E$$

Similarly

$$d_1' = DB \oplus (01 \cdot 13) \oplus (03 \cdot 53) \oplus 45 = 4D$$

$$d_2' = DB \oplus 13 \oplus (02 \cdot 53) \oplus (03 \cdot 45)$$

$$= A1$$

$$d_3' = (03 \cdot DB) \oplus 13 \oplus 53 \oplus (02 \cdot 45)$$

$$= B4$$

$$\begin{bmatrix} DB \\ 13 \\ 53 \\ 45 \end{bmatrix} \rightarrow \begin{bmatrix} 8E \\ 4D \\ A1 \\ B4 \end{bmatrix}$$

IT22038

Question 20: Critically evaluate AES's resistance to side-channel attacks, specifically timing analysis (e.g. cache-timing attacks on table based S-box implementation) and differential power analysis. Describe at least two concrete countermeasures (e.g. constant-time S-box; masking) and their performance trade-offs.

Solution:

AES is mathematically strong, but an implementation through physical side channel.

1. Timing / Cache attacks

Table based AES implementations often store S-box values in memory.

An attacker may be measure how long encryption takes or observe cache behavior.

Different secret dependent table accesses can produce different cache stats and timing patterns.

IT22028

AES algorithm secure $\neq$ implementation ultimately secure

Counter measure: Constant-time implementation

- AES - NI Hardware instructions
- bitsliced AES
- constant time implementations

Trade off:

- additional random-number generation
- extra memory
- Additional computation
- Implementation complexity

Therefore AES itself remains a strong cipher, but secure implementation requires protection against implementation level leakage.

IT22038

Question 23: Derive the RSA Key generation, encryption and decryption procedures from Euler's Theorem, and prove correctness of decryption (i.e., that $(m^e)^d \equiv m \pmod{n}$) for the general case $\gcd(m, n) \neq 1$.

Solution:

Key generation

1. Choose two large primes p, q

2. Compute

$$n = pq$$

3. Compute Euler's totient

$$\phi(n) = (p-1)(q-1)$$

4. Choose e such that

$$1 < e < \phi(n), \gcd(e, \phi(n)) = 1$$

5. Compute d such that

$$ed \equiv 1 \pmod{\phi(n)}$$

Therefore

- Public key (e, n)

- Private key (d, n)

Encryption

For message M :

$$c = M^e \pmod{n}$$

Decryption

$$M' \equiv c^d \pmod{n}$$

Thus $M' \equiv (M^e)^d = M^{ed} \pmod{n}$

since $ed = 1 + k\phi(n)$

for some integer k ,

$$M^{ed} = M^{1+k\phi(n)}$$

Case 1: $\gcd(M, n) = 1$

$$M^{\phi(n)} \equiv 1 \pmod{n}$$

Hence $M^{ed} = M (M^{\phi(n)})^k \equiv M (1)^k \equiv M \pmod{n}$

Case 2:

$$M^{ed} \equiv 0 \equiv M \pmod{p}$$

$$M^{p-1} \equiv 1 \pmod{p}$$

Chinese remainder theorem

$$M^{ed} \equiv M \pmod{n}$$

Question 21: Given primes $p=11$, $q=13$, generate a full RSA keypair (choosing a valid public exponent e) then sign a message Hash $H(m)$ and verify the signature, showing all intermediate modular exponentiation steps.

Solution:

Given $p=11$, $q=13$

calculate n and $\phi(n)$

$$n = pq = 11 \times 13 = 143$$

$$\phi(n) = (11-1) \times (13-1) = 10 \times 12 = 120$$

choose $e=7$

$$\gcd(7, 120) = 1$$

$$7d \equiv 1 \pmod{120}$$

$$7 \times 103 = 721 = 6(120) + 1$$

$$\therefore d = 103$$

Therefore Public key $= (7, 143)$

Private key $= (103, 143)$

IT22038

Step 2: $\text{sign}(H(m)) = 23$

$$S = H(m)^d \pmod{n}$$

$$S = 23^{103} \pmod{142}$$

Using repeated squaring

$$23^2 \equiv 100 \pmod{142}$$

$$23^4 \equiv 100^2 \equiv 42 \pmod{142}$$

$$23^8 \equiv 42^2 \equiv 74 \pmod{142}$$

$$23^{16} \equiv 74^2 \equiv 89 \pmod{142}$$

$$23^{32} \equiv 89^2 \equiv 65 \pmod{142}$$

$$23^{64} \equiv 65^2 \equiv 78 \pmod{142}$$

Since $103 = 64 + 32 + 4 + 2 + 1$

$$23^{103} \equiv 78 \cdot 65 \cdot 42 \cdot 100 \cdot 23 \equiv 23 \pmod{142}$$

$$\therefore S = 23$$

IT22038

Question 23 Derive a general formula for IPSec ESP overhead percentage in tunnel mode as a function of payload size, header size, authentication data size, padding, then evaluate 1200 byte payload with an 8-byte ESP header, 16 bytes of authentication data.

Solution

let P = payload size

H = ESP header size

A = authentication data size

D = padding size

$$O = H + A + D$$

$$T = P + H + A + D$$

$$\text{Overhead} = \frac{H + A + D}{P} \times 100$$

$$P = 1200 \text{ bytes}$$

$$H = 8, A = 16, D = 2$$

$$O = 8 + 16 + 2 = 26 \text{ bytes}$$

$$T = 1200 + 26 = 1226 \text{ bytes}$$

IT22038

$$\text{Overhead} = \frac{26}{1200} \times 100$$

$$= 2.17\%$$

Scaling:

$$\frac{26}{100} \times 100 = 26\%$$

For a large 10,000 byte payload

$$\frac{26}{10000} \times 100 = 0.26\%$$

Large payload $\Rightarrow$ smaller percentage overhead

Q24: Diffie - Hellman

Diffie - Hellman key exchange

$$p = 23, g = 5$$

$$a = 6$$

$$b = 15$$

$$A = g^a \text{ mod } p$$

$$A = 5^6 \text{ mod } 23$$

$$5^2 = 25 \equiv 2$$

$$5^4 \equiv 2^2 = 4$$

$$5^6 \equiv 5^4 \cdot 5^2 \equiv 4 \cdot 2 = 8$$

$$A = 8$$

IT22038

Bob's public value = $B = g^b \pmod p$

$$B = g^b \pmod p$$

$$B = 5^{15} \pmod{23}$$

$$5^1 \equiv 5, 5^2 \equiv 4, 5^4 \equiv 16, 5^8 \equiv 2$$

$$15 = 8 + 4 + 2 + 1$$

$$5^{15} \equiv 16 \cdot 4 \cdot 2 \cdot 5 \equiv 19 \pmod{23}$$

$$B = 19$$

Shared secret:

$$K_B = A^b \pmod p$$

$$K_B = 5^{15} \pmod{23} = 19$$

$$K_B = 2$$

$$K_A = K_B = 2$$

IT22038

Q25: Compare the computational cost of establishing a shared key via RSA key transport versus Diffie-Hellman key agreement at an equivalent security level (e.g. 3072-bit RSA/DH versus 256 bit ECDH). Discuss why forward secrecy favors (EC)DHE over RSA key transport in modern TLS.

Solution:

At approximately 128 bit classical security, typical examples are

Scheme	Example key size
RSA	3072 bits
finite field DH	3072 bits
ECDH	256 bits

computational cost

RSA uses large-integer arithmetic. A 3072-bit RSA key involves relatively large modular exponentiation.

Finite Field DH also uses large modulus exponentiation with approximately 3072 bits numbers.

Forward secrecy

With traditional RSA key transport the client encrypts a session secret using the server's RSA public key. If the server's long term private RSA key is later compromised and recorded, traffic is available, past sessions may be recoverable.

With ephemeral Diffie-Hellman

ECDEH/DEH provides forward secrecy

Q26:

In an ElGamal scheme with $p = 467$, $g = 2$, private key $x = 127$, message hash $H(m) = 100$ and nonce $k = 213$ (with $\gcd(k, p-1) = 1$): Compute the public key, generate the signature (r, s) and verify it.

Solution:

Given

$$p = 467, g = 2, x = 127$$

$$H(m) = 100, k = 213$$

$$\text{since } \gcd(213, 466) = 1$$

the nonce is valid.

Step 1:

Public Key

$$y = g^x \bmod p$$

$$y = 2^{127} \bmod 467$$

$$y = 132$$

$$(p, a, y) = (467, 2, 132)$$

$$r = g^k \bmod p$$

$$r = 2^{213} \bmod 467$$

IT22038

$$r = 29$$

Find k^{-1}

$$213 \cdot k^{-1} \equiv 1 \pmod{466}$$

$$k^{-1} = 431$$

$$213(431) \equiv 1 \pmod{466}$$

Calculate s:

$$s = k^{-1} (H(m) - xr) \pmod{p-1}$$

$$s = 431 (100 - 127(29)) \pmod{465}$$

$$127(29) = 3683$$

$$100 - 3683 = -3583$$

Question 27: Prove algebraically that the El Gamal signature verification $g^{H(m)} \equiv y^r s \pmod{p}$ holds for a correctly generated signature, starting from the definition $s \equiv k^{-1} (H(m) - xr) \pmod{p-1}$

$$(g^{H(m)})^k \equiv (y^r s)^k \pmod{p}$$

$$g^{H(m)} \equiv y^r s \pmod{p}$$

$$g^{H(m)} \equiv y^r s \pmod{p}$$

solution:

Given

$$s \equiv k^{-1} (H(m) - xr) \pmod{p-1}$$

multiply by k :

$$ks \equiv H(m) - xr \pmod{p-1}$$

Therefore

$$H(m) \equiv xr + ks \pmod{p-1}$$

$$g^{H(m)} = g^{xr + ks} \pmod{p}$$

$$g^{H(m)} = (g^x)^r (g^k)^s \pmod{p}$$

$$y = g^x \pmod{p}$$

$$r = g^k \pmod{p}$$

$$g^{H(m)} = y^r r^s \pmod{p}$$

Question no: 28

On the elliptic curve $y^2 \equiv x^3 + x + 1 \pmod{23}$ verify that $P = (3, 10)$ lies on the curve, compute $2P$ via point doubling, and compute $P + Q$ for $Q = (9, 7)$, showing the full modular arithmetic derivation of the slope in each case.

Solution:

Curve: $y^2 \equiv x^3 + x + 1 \pmod{23}$

Thus $a = 1$

verify $P = (3, 10)$

left side: $y^2 \equiv 10^2 \equiv 100 \equiv 8 \pmod{23}$

Right side: $x^3 + x + 1 \equiv 3^3 + 3 + 1 \equiv 31 \equiv 8 \pmod{23}$

Therefore $P = (3, 10)$ lies on the curve

Calculate $2P$

For doubling

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{23}$$

IT22038

Substitute

$$\lambda = \frac{3(3)^2 + a}{2(10)} = \frac{28}{20}$$

Modulo 23:

$$28 \equiv 5, \quad 20^{-1} \equiv 15$$

$$20(15) = 300 \equiv 1 \pmod{23}$$

$$\lambda \equiv 5(15) = 75 \equiv 6 \pmod{23}$$

$$x_3 = \lambda^2 - 2x_1$$

$$x_3 = 6^2 - 6 = 30 \equiv 7$$

$$y_3 = \lambda(x_1 - x_3) - y_1$$

$$y_3 = 6(3 - 7) - 10 = -34 \equiv 12$$

$$2P = (7, 12)$$

Calculate P+Q:

$$Q = (9, 7)$$

$$\lambda = \frac{7 - 10}{9 - 3} = \frac{-3}{6} \pmod{23}$$

$$6^{-1} \equiv 4 \pmod{23}$$

$$\lambda \equiv (-3)(4) = -12 \equiv 11$$

Now,

$$x_3 = 1^2 - x_1 - x_2$$

$$x_3 = 1^2 - 3 - 9 = -10 \equiv 17 \pmod{23}$$

$$y_3 = 1(x_1 - x_3) - y_1$$

$$y_3 = 1(3 - 17) - 10 = -16 \equiv 20 \pmod{23}$$

$$P + Q = (17, 20)$$

Question 29: On the curve $y^2 \equiv x^3 + 7x + 10 \pmod{37}$ with base point $G(2, 5)$ of order $n = 19$ and private key $d = 9$, compute the public key $Q = dG$, generate an ECDSA signature (r, c) for $H(m) = 8$ using nonce $k = 3$, and verify it against Q .

Solution

$$\text{Curve } y^2 \equiv x^3 + 7x + 10 \pmod{37}$$

$$\text{Given, } G = (2, 5), n = 19, d = 9$$

$$H(m) = 8, k = 3$$

$$\text{Public key } Q = dG = 9G$$

IT22038

$$Q = (2, 3, 11)$$

Calculate kG

$$kG = 3G$$

$$3G = 28, 5$$

$$r = x_3 G \bmod n$$

$$r = 28 \bmod 19$$

$$r = 9$$

Calculate s :

ECDSA:

$$s = k^{-1} (H(m) + dr) \pmod{n}$$

$$3^{-1} \equiv 13 \pmod{19}$$

$$3(13) = 39 \equiv 1 \pmod{19}$$

$$s = 13(8 + 9 \times 9) \pmod{19}$$

$$= 13(89) \pmod{19}$$

$$89 \equiv 13 \pmod{19}$$

$$s \equiv 13(13) = 169 \equiv 17 \pmod{19}$$

$$(r, s) = (9, 17)$$

Step 4: Verification :

Compute,

$$w = s^{-1} \pmod{n}$$

$$17^{-1} \equiv 9 \pmod{19}$$

$$w = 9$$

Then

$$u_1 = H(m)w \pmod{n}$$

$$u_1 = 8(9) = 72 \equiv 15 \pmod{19}$$

$$u_2 = rw \pmod{n}$$

$$u_2 = 9(9) = 81 \equiv 5 \pmod{19}$$

$$x = u_1G + u_2Q$$

$$x = 15G + 5Q$$

$$x = (22, 36)$$

$$v = x_x \pmod{n}$$

$$v = 22 \pmod{19}$$

$$v = 3$$

Question 30: Compare the key size security equivalence between RSA, ElGamal, and ECC (e.g. 256 bit ECC $\approx$ 3072 bit RSA/ElGamal) at the ~ 128 bit security level)

Solution:

Approximate classical security equivalence

Cryptosystem	Example key size	Approx. security
RSA	3072-bit	~ 128 -bit
ElGamal	~ 3072 -bit	~ 128 -bit
ECC	256 bit	~ 128 bit

The key size difference is mainly due to the different mathematical problems underlying the systems.

RSA and traditional ElGamal rely on problems involving large integer arithmetic such as

integer factorization, or discrete logarithms in finite fields.

Question no: 31

Formally define preimage resistance, second-preimage resistance and collision resistance for a cryptographic hash function. Derive the birthday attack complexity ($\approx 2^{n/2}$) for finding a collision in an n bit hash, and explain why this motivates 256-bit (rather than 128-bit) output lengths for modern hash functions.

Solution:

Preimage Resistance

A hash function is preimage resistant, if given a hash value h , it is computationally infeasible to find any message m such that,

$$H(m) = h$$

For an n -bit hash, a generic brute-force attack requires approximately

2^n hash evaluations

(b) Second Preimage Resistance

Given a particular message m_1 , it should be computationally infeasible to find a different message $m_2 \neq m_1$ such that

$$H(m_1) = H(m_2)$$

So, $m_1 \neq m_2$ but $H(m_1) = H(m_2)$

It also requires approximately 2^n operation for a generic n -bit hash

(c) Collision Resistance

It should be computationally infeasible to find any two different messages m_1, m_2 such

that $m_1 \neq m_2$

$$H(m_1) = H(m_2)$$

For an n -bit hash, generic collision search requires approximately $2^{n/2}$ operations.

Birthday attack derivation:

An n -bit hash has

$N = 2^n$ possible hash values

Suppose we randomly hash k different messages

The probability that there is no collision is approximately $p(\text{no collision}) \approx e^{-k(k-1)/(2N)}$

A collision becomes likely when

$$\frac{k(k-1)}{2N} \approx 1$$

Therefore $k^2 \approx 2N$

since $N = 2^n$

We get $k \approx \sqrt{2(2^n)}$

$$k \approx \sqrt{2} \cdot 2^{n/2}$$

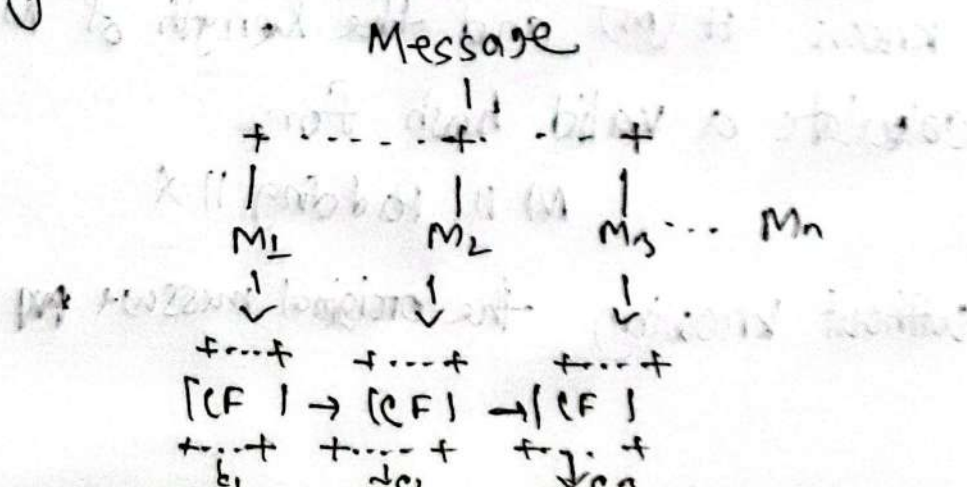
Question no 32

Trace the Merkle-Damgård construction with a diagram showing message blocks, compression function calls, and chaining variables. Explain the length extension attack that this construction is vulnerable to, and how HMAC and SHA-3 (Sponge construction) avoid it.

Solution:

Merkle-Damgård construction

The Merkle Damgård construction builds a hash function by processing the message in fixed size blocks.

Diagram

IT22038

Where

IV = Initial value

M_i = message block

CF = Compression function

C_i = chaining variable

Final C_n = hash output

The basic operation is

$$C_i = F(C_{i-1}, M_i)$$

with $C_0 = IV$

and finally $H(M) = C_n$

Length Extension Attack

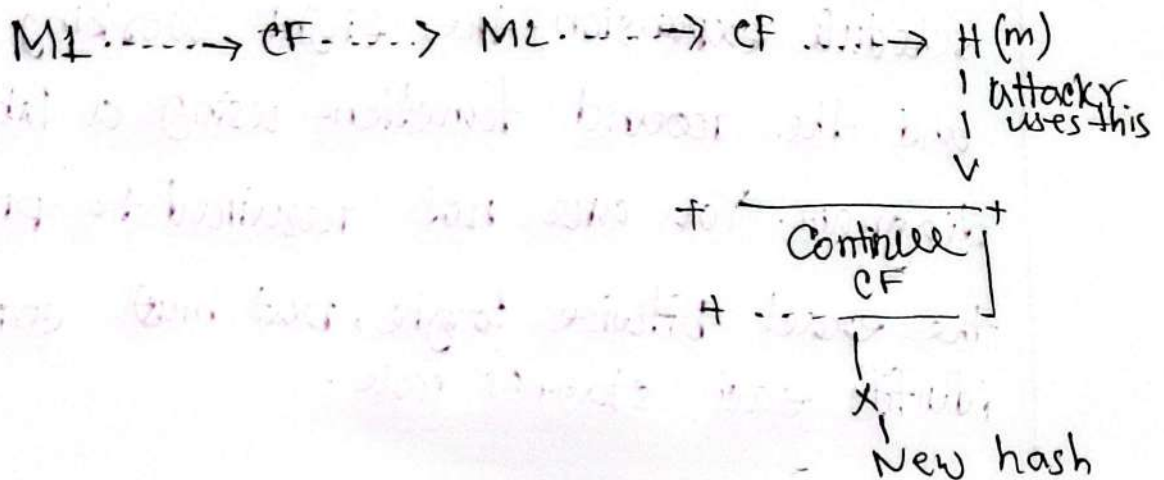
Many Merkle-Damgård hash functions have a property that allows an attacker who knows $H(M)$ and the length of M , to calculate a valid hash for

$$M \parallel \text{Padding} \parallel X$$

without knowing the original message ~~M~~

Concept

Original



The attacker treats the known hash $H(m)$ as the original chaining state and continues hashing additional data x .

Thus

$$H(m) \rightarrow H(m \parallel \text{padding} \parallel x)$$

Can be computed without knowing m ,
provided its length is known

IT22038

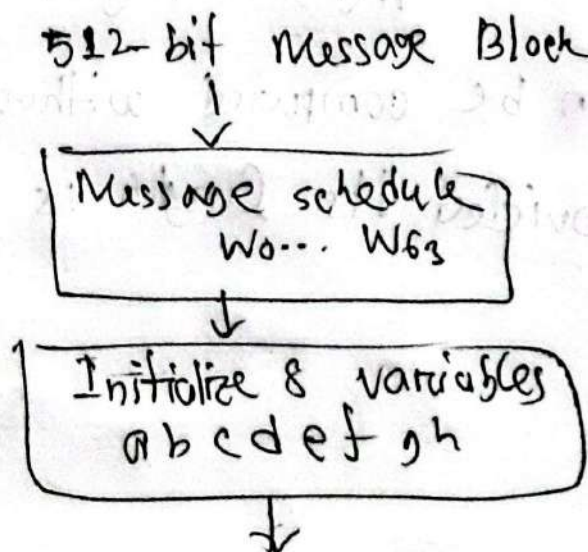
Question 33: Explain the high-level internal structure of the SHA-256 compression function — message schedule expansion, the eight working principle and the round function — using a labelled block diagram. You are not required to produce the exact bitwise logic, but must correctly identify each stage's role.

Solution

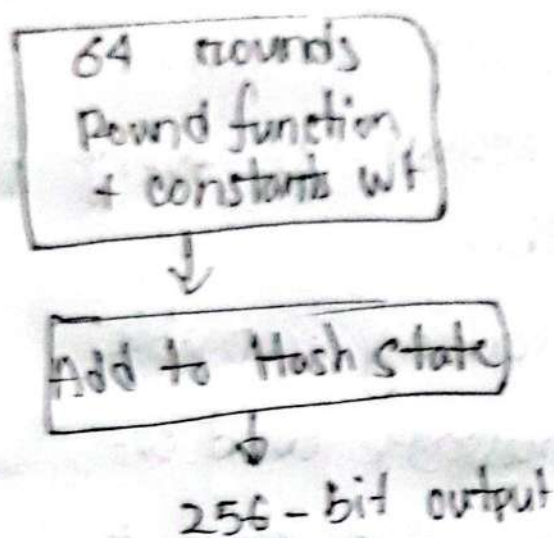
SHA-256 processes each 512 bit message block and produces a 256 bit hash state.

The compression function performs 64 rounds

High Level Diagram



IT22038



1. Message schedule.

The original 512 bit block is divided into 16 words each of 32 bits

so initially $w_0, w_1, \dots, w_{15}$ are obtained.

These are expanded to $w_0, w_1, w_2, \dots, w_{63}$.

2. Eight working variable

SHA-256 maintains eight 32 bit working variable

a, b, c, d, e, f, g, h

3. Round function

64 rounds, In every round, the algorithm combines

- working variables
- message word w_r
- round constant K_r
- logical/bitwise operations

Question 34: Formally define the Shortest Vector Problem (SVP) and the Shortest Independent Vectors Problem (SIVP) in lattice theory, and explain why both are believed to remain hard even for quantum computers.

Solution:

A lattice is a set of points generated by integer combinations of basis vectors

For basis vectors

$$b_1, b_2, \dots, b_n$$

IT22038

The lattice is

$$L(B) = \left\{ \sum_{i=1}^n z_i b_i : z_i \in \mathbb{Z} \right\}$$

① Shortest Vector problem (SVI)

Given a lattice L , find a non zero lattice vector of minimum length.

Formally

$$v^* = \arg \min_{v \in L, v \neq 0} \|v\|$$

The length of the shortest vector is often written as

$$\lambda_1(L) = \min_{v \in L, v \neq 0} \|v\|$$

SIVP - Shortest Independent Vectors problem.

Given an n -dimensional lattice L , find n linearly independent vectors whose lengths are as short as possible

One formulation defines $\lambda_n(L)$

as the minimum radius r such that there exist n linearly independent lattice vector $v_1, \dots, v_n$ satisfying $\|v_i\| \leq r$

However there is no known quantum algorithm analogous to Shor's algorithm that efficiently solves general purpose cryptographic lattice problems such as SVP or the lattice problems underlying LWE/Module LWE.

No efficient quantum algorithm is currently known for solving the relevant hard lattice problems at cryptographic dimensions.

IT22038

Question 35 Explain the core idea behind CRYSTAL - Kyber's Module Learning with Errors (Module LWE) key encapsulation mechanism, including how noise is used to hide the secret, and why this construction resists both classical and quantum attacks.

Solution

Important terminology: CRYSTALS - Kyber was selected by NIST and standardized as ML-KEM. In an exam question that says Kyber,

Basic idea: Kyber is a lattice based Encapsulation Mechanism (KEM)

Its security based on the difficulty of problems related to Module LWE (Learning with errors)

Suppose we have

$$b = As + e$$

Where,

A = public matrix

s = secret vector

e = small random error/noise

b = resulting vector

Without noise, an attacker could potentially solve

$$b = As$$

Module LWE

Kyber uses a structured version called Module LWE,

Instead of ordinary vectors over simple finite field, Module-LWE works with polynomial modules over suitable rings

At a high level:

$$b = As + e$$

Where

A = public module matrix

s = secret

e = small noise / error

t = public value

RSA and ECC depend on problem that
Shor's algorithm can solve efficiently
on a sufficiently powerful quantum computer

Currently, no efficient quantum algorithm
is known that breaks Module-LWE in the
way Shor's algorithm breaks factoring
and discrete logarithms

Module-LWE $\rightarrow$ Lattice hardness $\rightarrow$ Post
quantum security

Question no: 36: Diagram and explain the full authentication protocol, covering the Authentication server (AS) exchanges, Ticket Granting Server (TGS) exchanges, and client-server (CS) exchanges. Include how session keys and ticket lifetimes are established and why the TGT avoids repeated password transmission.

Solution

Kerberos is a network authentication protocol that provides secure authentication using ticket session keys, timestamps and a trusted third party. It allows a client to access multiple services without repeatedly sending the password.

Main components

- Client (C): User or client machine
- Authentication Server: Authenticates the client
- Ticket Granting server: Issues tickets for individual services

Step 1: Authentication Server (AS) Exchange

The client sends:

$$C \rightarrow AS: ID_C, ID_{DAS}$$

The AS generates a session key

$$K_{e,AS}$$

This key will be used between the client and TAS. The AS also create a Ticket Granting Ticket (TGT)

$$TGT = \{ ID_C, ID_{TAS}, ID_{DAS}, K_{e,AS}, Lifetime \}_{K_{e,AS}}$$

Step 2: TAS Exchange

When the clients want to access services, it sends

$$C \rightarrow TAS: TGT, Authenticator, ID_S$$

The authenticator may contain

$$Authenticator = \{ ID_C, Timestamp \}_{K_{e,TAS}}$$

Step 3: Client server Exchange

The client sends :

$C \rightarrow S$: Tickets, Authenticator

The service server decrypts the ticket and verifies the authenticator.

For mutual authentication, the server returns

$S \rightarrow C$: $\{ \text{Timestamp} + 1 \}_{K_{cs}}$

The client's password is not repeatedly sent over the network. After the initial authentication the client stores the TGT and K_{TAS} . It can use the TGT to obtain service tickets for different services.

Therefore,

Password $\rightarrow$ Initial authentication only

TGT $\rightarrow$ Obtain service tickets.

Question no: 37: Explain why a plain hash-based MAC (i.e. $H(k || M)$) is vulnerable to a length-extension attack, and derive how HMAC's nested construction $H(k \oplus \text{ipad}, H(k \oplus \text{opad}, M))$ eliminates this vulnerability. Illustrate with a small worked (simplified numeric) MAC example.

Solution:

A simple hash-based MAC can be constructed as $MAC = H(k || M)$

where k is the secret key and M is the message. For some M'

$$M^* = M || \text{padding} || M'$$

with a corresponding valid MAC

HMAC solution:

HMAC uses a nested construction

IT22038

Day: _____

Time: _____

Date: ____/____/____

$$H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel M))$$

Inner hash:

$$H_{\text{inner}} = H((K \oplus \text{ipad}) \parallel M)$$

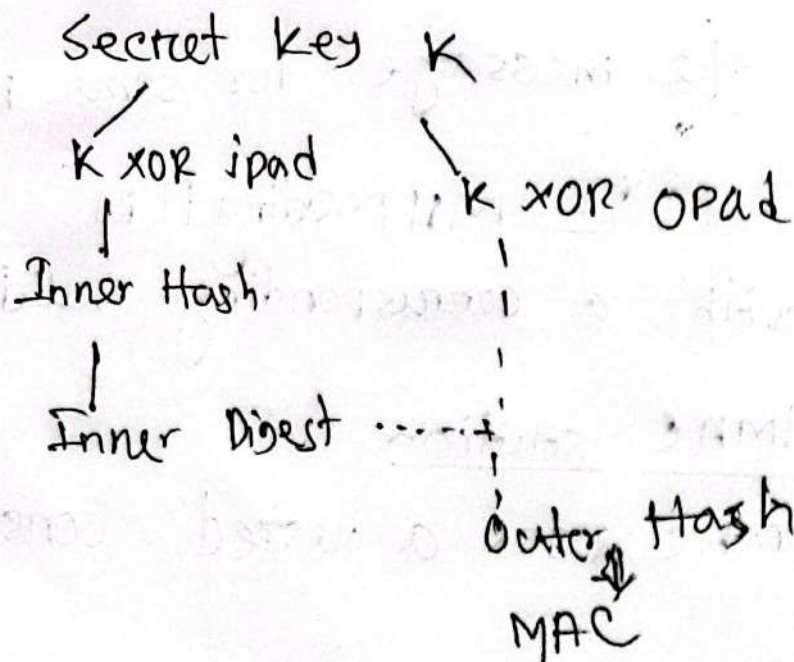
Outer hash:

$$\text{MAC} = H((K \oplus \text{opad}) \parallel H_{\text{inner}})$$

Where

- ipad = inner padding
- opad = outer padding
- K = secret key

structure



The attacker cannot simply extend the inner message because the outer hash is keyed separately.

Therefore, HMNC prevents the length extension problem of the simple $H(K || M)$ construction.

Simplified Numerical Example

Consider a toy hash

$$H(x) = x \bmod 100$$

This is, not a real cryptographic hash. It is only for illustration.

$$K = 23, M = 'A'$$

$$MAC = H(K || M)$$

$$= H(23 || 'A')$$

$$= H(64)$$

$$MAC = 64$$

Question 38: Diagram the TLS 1.3 handshake (ClientHello / ServerHello with key shares, encrypted extensions, finished message) and explain how the session keys are derived via HKDF from the (EC) DHE shared secret. Compare this against the TLS 1.2 handshake in terms of round trip count and forward secrecy.

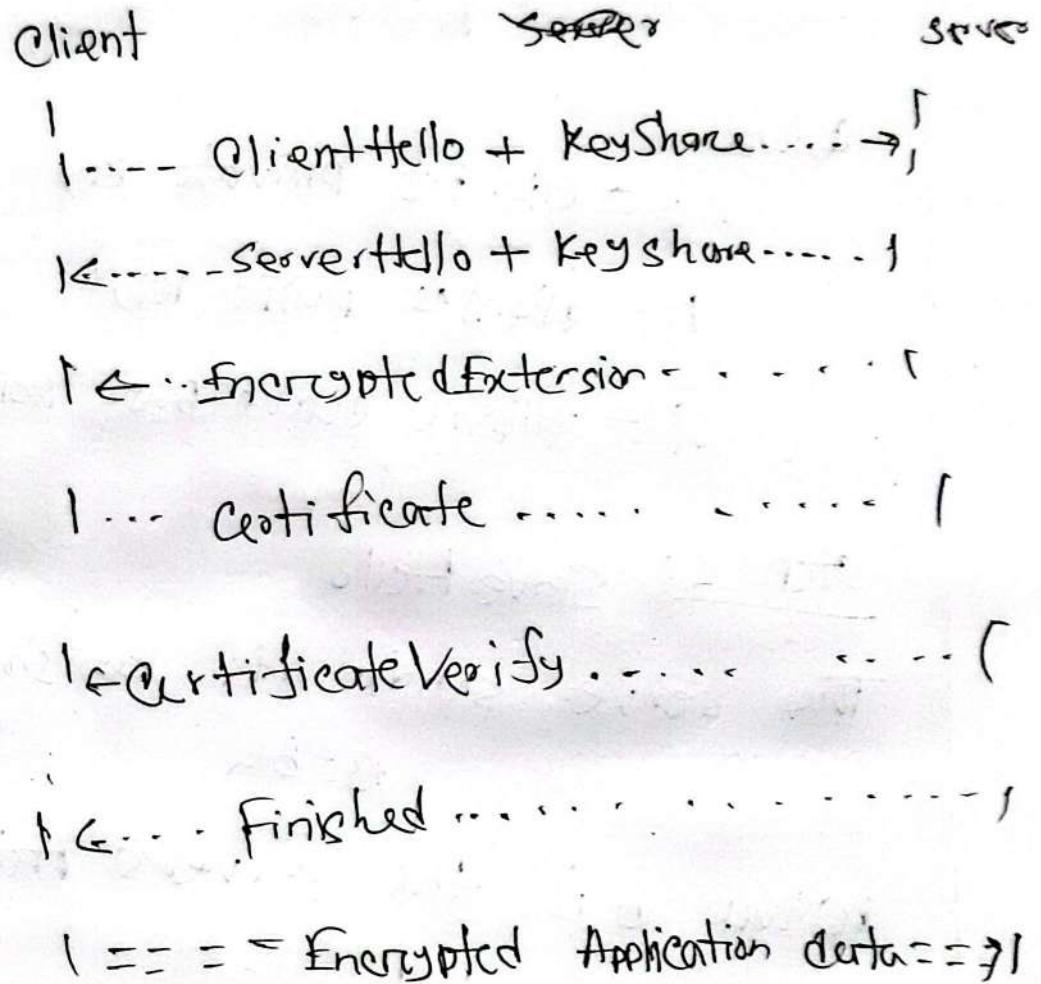
Solution:

TLS (Transport Layer Security) provides

1. Authentication
2. Confidentiality
3. Integrity

TLS 1.3 uses and ephemeral (EC) DHE Key exchange to establish a shared secret

TLS 1.3 Handshake.



Step 1: ClientHello

The client sends

- supported TLS version
- cipher suites
- Random value
- key share

for example with ECDFE:

$$A = aG$$

Where

a = client's private key

A = client's public key

G = elliptic curve base point

step 2: Server Hello

The server sends its key share

$$B = bG$$

Where b is the server's private key

client calculates

$$Z = aB = abG$$

$$Z_{\text{client}} = Z_{\text{server}}$$

Question 39: IoT Bandwidth, Design a bandwidth and duty cycle budget for an IoT sensor network in which each node transmits a 1200-byte packet once per minute over a LPWAN technology of your choosing (e.g., LoRaWAN). Compute the data volume per node per day (in bytes and KB) and discuss regular duty to cycle constraints.

Solution:

Given

Each IoT sensor transmits

1200 bytes / packet

Frequency

1 packet / minute

IT22038

Day: _____

Time: _____

Date: / /

1. Packets per day

There are $80 \times 24 = 1920$ minutes/day

2. Data per day

Each packet contains 1200 bytes

$$\text{Data/day} = 1200 \times 1920$$

$$= 2,304,000 \text{ bytes/day}$$

Therefore

$$2,304,000 \text{ bytes/day}$$

$$\text{Using } 1 \text{ KB} = 1024 \text{ bytes}$$

$$\text{KB} = \frac{2,304,000}{1024}$$

$$2250 \text{ KB/day}$$

$$2.25 \text{ MB/day}$$

Day: _____

Time: _____ Date: / /

IT22038

Average Data Rate:

$$\text{Rate} = \frac{1200 \times 8}{60}$$

$$= \frac{9600}{60}$$

$$= 160 \text{ bps}$$

Duty cycle constraint

suppose a regulatory band imposes 1% duty cycle.

Duty cycle means the transmitter may occupy the radio channel for only a limited fraction of time

$$1\% = \frac{1}{100}$$

$$3600 \times 0.01 = 36 \text{ second}$$

$$36 \text{ seconds/hour}$$

$$86400 \times 0.01 = 864 \text{ second/day}$$

Question 40: Critically assess lightweight cryptographic standard suitable for resource constrained IoT devices (e.g. NIST, Aseon selected as the lightweight cryptography standard) against AES 128 in terms of memory footprint, energy consumption, and side-channel resistance. Recommend which is more appropriate for a battery powered sensor with < 10 KB of RAM.

Solution

Aseon is a lightweight cryptographic family standardized by NIST for constrained devices. AES-128 is a widely deployed 128 bit block cipher.

Both can provide strong security, but their implementation characteristics differ.

17/2/2038

Day: _____
Time: _____ Date: / /

Feature	Ascon	AES-128
Design goal	Lightweight/ IoT oriented	General purpose encryption
Block / state structure	320 bit internal state	128 bit block
Key size	Commonly 128 bit	128 bit
Memory	Designed for constrained implementation	Can require more implement resource depending on implementation

Memory Footprint

Ascon was designed specifically for constrained environments and can be implemented with relatively small ram and code requirements

- Look up-table implementation
- Bitsliced implementation
- Hardware acceleration

Energy consumption

For a battery powered sensor, energy efficiency is important.

Aseon's lightweight design can make it attractive for constrained devices.

However if a microcontroller has hardware AES acceleration.

Side channel resistance

Neither Aseon nor AES is automatically side channel resistant.

An implementation must protect against

- Timing attacks
- Power analysis
- Electro magnetic leakage
- Fault attacks.